Parallelisation of Lattice Boltzmann Solver

MATH-454 Parallel and High Performance Computing

Ahmed Rockey Saikia 416140

1. Introduction

The Lattice Boltzmann Method (LBM) is a kinetic theory based solver that evolves a discrete velocity distribution function on a grid. This report describes the parallelisation of a serial D2Q9 LBM solver simulating 2D flow past a circular cylinder at Reynolds number $\mathrm{Re} = 100$. Each time step executes the pipeline in the order `collide` → `bounce-back` → `stream` → `apply inlet` → `apply outlet`. The Strouhal number of the periodic vortex pattern, the Karman vortex street, $\mathrm{St} \approx 0.16$, validates correctness for the parallel implementations.

2. Serial Profiling

Serial profiling with `perf` on the JED cluster (Intel Xeon Platinum 8360Y @ 2.40 GHz, single core, `-O3`) shows that `collide()` and `stream()` together consume 98.2% of all CPU cycles (Table 1). The fact that the mega lattice updates per second (MLUPS) is approximately constant for different grid sizes (Table 1) suggests that the performance is memory-bound. This is justified by the profiling data showing that `stream()`, which contains zero floating-point operations, account for 26% of total cycles. Two parallel implementations are discussed: MPI for distributed memory multi-node scaling and CUDA for GPU acceleration.

(a) Serial baselines

Grid Size	Cells	MLUPS
800×400	320 K	24.17
1600×800	1.28 M	23.73
3200×1600	5.12 M	23.65

(b) `perf` results (800×400)

Function	% Cycles
`collide()`	71.78
`stream()`	26.45
`bounce_back()`	1.40
`apply_inlet()`	0.27

Table 1: Serial baselines on JED (Xeon 8360Y, single core, `-O3`) and `perf` breakdown of functions

3. MPI Implementation

A 1D slicing of the computational domain assigns each MPI rank a contiguous strip of $n_{x,\mathrm{local}}$ columns spanning the full n_y rows. A single auxiliary column is maintained on each side and exchanged with neighboring ranks via two `MPI_Sendrecv` calls per step. Because the distribution functions use a structure of arrays (SoA) memory layout, the nine velocity components are non-contiguous in memory. So each auxiliary layer is packed into a $9 \times n_y$ buffer before transmission and unpacked on receiving it.

Rank 0 applies the inlet boundary condition ($x = 0$) while the last rank applies the outlet. The top and bottom no-slip wall conditions are handled locally by every rank. To profile the step loop performance, we use `MPI_Barrier` and `MPI_Wtime`, followed by an `MPI_Reduce(MPI_MAX)` operator to report the worst-case wall time. Because the `apply_inlet` function (0.27% of total cycles) is strictly executed by Rank 0, the theoretical Amdahl serial fraction is limited to $s \approx 0.0028$. Finally, parallel correctness was ensured at $P = 4$ processes by confirming a Strouhal number of $\mathrm{St} = 0.1600$.

4. MPI Results

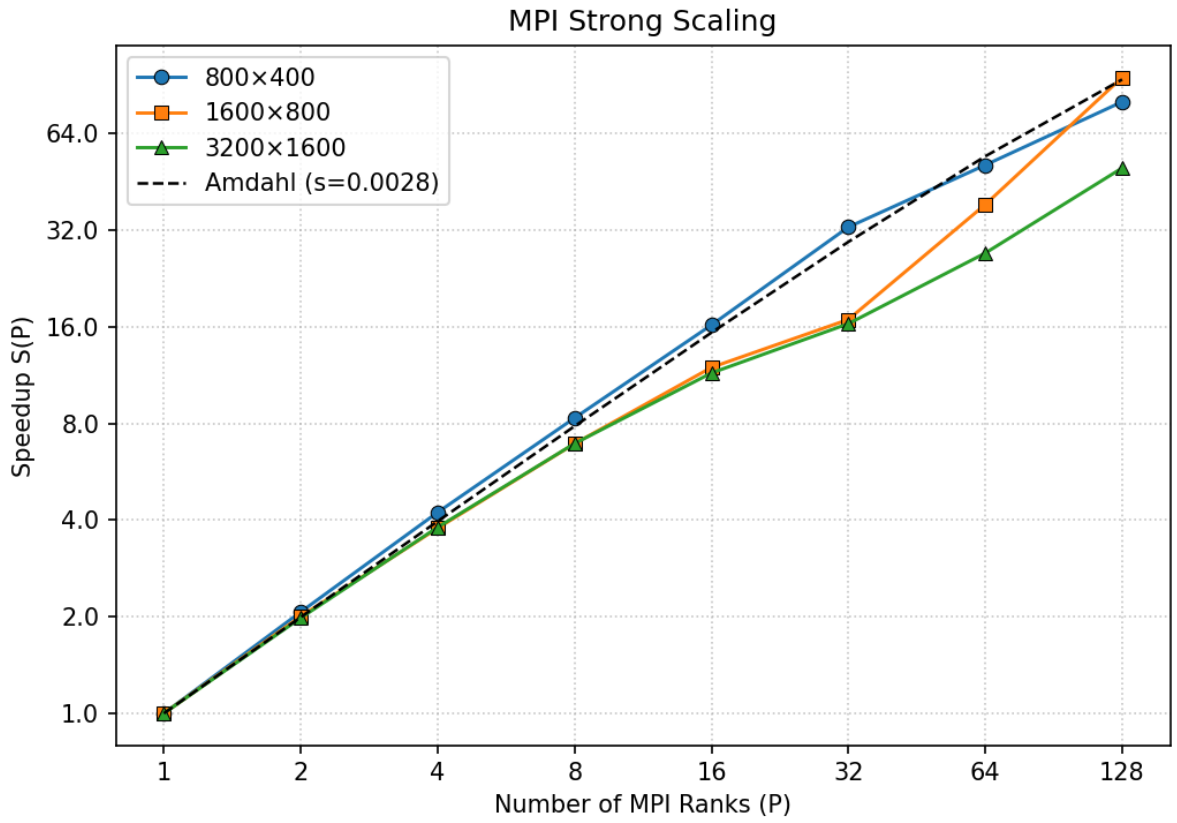


Figure 1: MPI strong scaling speedup $S(P)$ for three grid sizes (800×400, 1600×800, 3200×1600 and Amdahl reference).

4.1 Strong Scaling

Strong scaling was measured for three grids over $P = 1$ to $P = 128$ ($P = 128$ across two nodes). These cases exhibit important qualitative behaviours (Figure 1)

800×400 With the 1D strips, each rank owns $n_{x,\text{local}} = n_x/P$ columns, and the auxiliary cells to compute cells ratio per rank is

$$\eta = \frac{2\,n_y}{n_{x,\text{local}}\,n_y} = \frac{2P}{n_x}. \tag{1}$$

At $P = 128$, $\eta = 32\%$, a third of each rank's working data must be exchanged every step. This growing communication over compute explains the sublinear speedup observed for the 800×400 grid at high P. We attribute communication bound here as 800×400 grid fits in the cache memory.

1600×800 Although scaling is sublinear overall ($S(P) < P$), efficiency drops to 52.7% at $P = 32$, then recovers to 60.1% at $P = 64$ and 74.9% at $P = 128$. This recovery stems from increase in cache hits. The data for each rank (nine distributions and double precision) is given by

$$D(P) = \frac{n_x}{P} \times n_y \times 9 \times 8\text{B} = \frac{92.2\text{MB}}{P} \tag{2}$$

At $P = 32$, the 2.88 MB the data spilts into cache and main memory. However, for ($P > 64$) the data size reduces to 1.44 MB at $P = 64$ and 0.72 MB at $P = 128$, allowing it to fit entirely within the 1.5 MB per core L3 cache of Xeon 8360Y.

3200×1600 This larger domain exhibits sublinear scaling with declining parallel efficiency monotonically to 39% at $P = 128$. At 368.6 MB, our experimented ranks remain below the cache

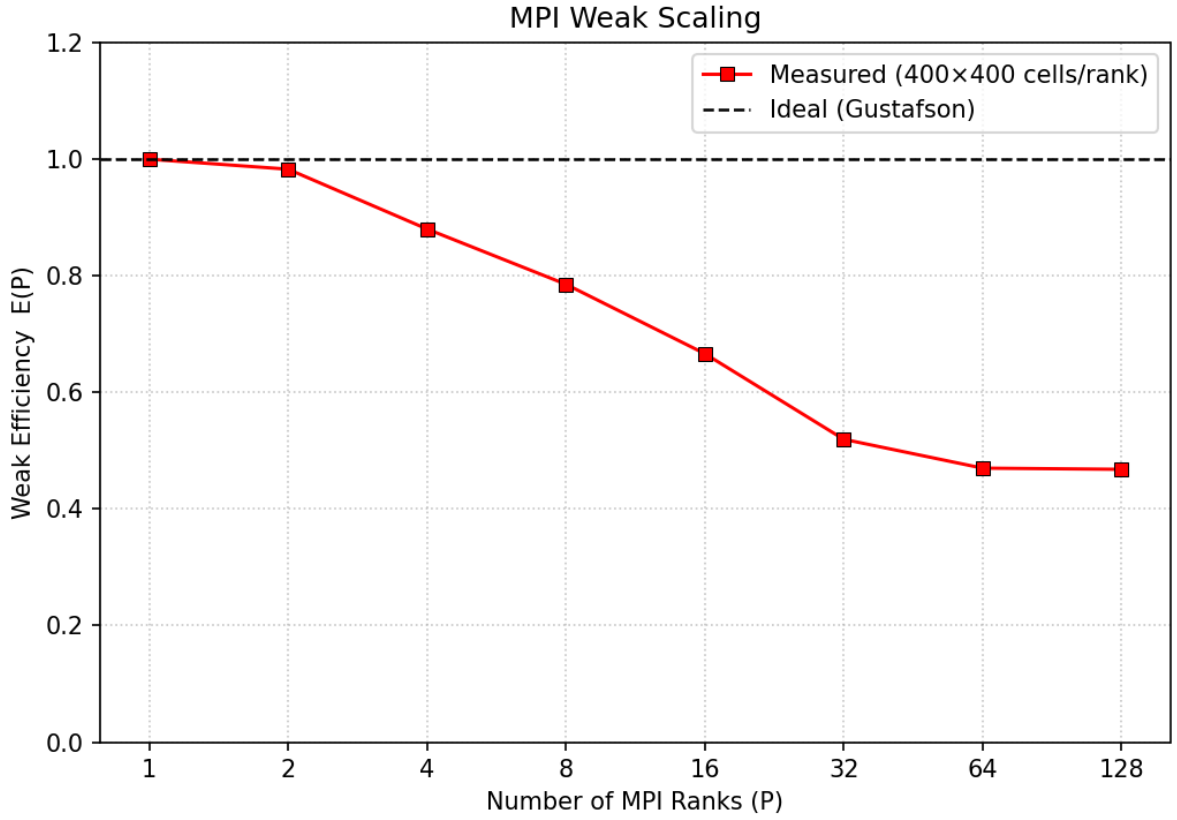


Figure 2: MPI weak scaling efficiency $E(P) = T(1)/T(P)$ for fixed 400×400 cells/rank.

residency threshold of 246 ($\approx 368.6/1.5$). The per rank data remains at 2.88 MB at $P = 128$, preventing it from fitting into the 1.5 MB L3 cache and leading to cache misses and decreased memory bandwidth. Therefore, the solver's performance is limited by memory bandwidths rather than any algorithmic improvement.

Amdahl reference. With $s = 0.0028$ (0.27% `apply_inlet` cycles), Amdahl's law predicts an upper bound of $S_\infty(128) = 94.4\times$. The 3200×1600 result ($S \approx 50$) lies well below this limit but the 1600×800 result exceeds this bound, which shows that cache effects are not considered in Amdahl's model.

4.2 Weak Scaling

Weak scaling fixes 160K cells per rank ($n_x = 400P$, $n_y = 400$) and measures efficiency $E(P) = T(1)/T(P)$ where the Gustafson ideal is $E = 1$. Efficiency declines monotonically from 100% at $P = 1$ to 47% at $P = 64$ as an increasing number of ranks compete for limited memory bandwidth. The plot shows that the efficiency drop after doubling ranks decreases from $32 \to 64$ which reflects the dual sockets of JED. Furthermore, the $P = 64 \to 128$ transition (scaling from one node to two) wall time remains almost constant (25.81 → 25.92s) while MLUPS doubles from 793 → 1580. The flat wall time across the node boundary confirms that per socket memory bandwidth contention causes the efficiency reduction observed on a single node (Figure 2).

5. CUDA Implementation

The CUDA solver uses Tesla V100-32GB with peak bandwidth of ≈ 900 GB/s on the Izar cluster. One CUDA thread is assigned per lattice cell, mapped onto a 2D thread grid. Memory layout is templated on a `Layout` enum selecting structure of array (SoA) or array of structure (AoS) at compile time. We experiment on two execution methods: *separate* kernels (`collide` →

`bounce-back` → `stream`) and a *combined* (`collide+bounce-back+stream`) kernel that eliminates the intermediate write and read, halving data transfer from 288 B/cell to 144 B/cell. To profile the code, we use CUDA events around the step loop with `cudaDeviceSynchronize()` before the start event. Further, validation of the code is confirmed for both kernels by obtaining the Strouhal number of St = 0.1600.

6. CUDA Results

(a) Block Analysis (4096×2048*)*

Block	MLUPS
8×32	1913
16×16	2371
32×8	2558
64×8	2588
128×4	2607

(b) Layout SoA vs AoS

Grid	SoA	AoS
1024×512	2281	643
2048×1024	2480	642
4096×2048	2558	638
8192×4096	2513	633

(c) Combined vs Separate Kernels

Kernel	B/cell	MLUPS	AI (FLOP/B)	GFLOP/s	BW (GB/s)
Separate	288	2513	0.64	465	724
Combined	144	3937	1.28	728	567

Table 2: CUDA performance on the V100 (SoA, 2000 steps). **(a)** `block_x` ≥ 32 is required for warp coalescing. **(b)** SoA outperforms AoS by $\sim 4\times$ **(c)** Roofline summary at 8192×4096 *combined* kernel halves data transfer (288 → 144 B/cell), raising MLUPS at every size.

6.1 Block Size Analysis

To evaluate the impact of memory access, we tested different block sizes at 4096×2048 using SoA layout. Configurations with `block_x` < 32 (e.g. 8×32) perform suboptimally with ~ 1913 MLUPS because threads in a warp have to access non-contiguous memory. This is confirmed by block sizes with `block_x` ≥ 32 which perform equivalently ($\sim$ 2588–2607 MLUPS). Hence, in the subsequent runs we use `block_x` $= 32$ for analysis.

6.2 Problem Size Analysis

To evaluate the performance on different problem sizes, we fixed the block configuration to 32×8 SoA layout. We varied the problem size from 512×256 to 8192×4096. MLUPS initially rises but quickly saturates to ~ 2500 (Table 2). This peak represents 80% of the theoretical bandwidth roofline of 3125 MLUPS ($= \frac{900\text{GB/s}}{288\text{B/cell}}$). Reaching a saturation limit, independent of problem size, indicates that the GPU is saturated.

6.3 Layout Analysis

SoA layout beats AoS layout by $\sim 4\times$ for all problem sizes as shown in Table 2. This gap is due to the fact that AoS groups all nine directional values for single cell together in memory. Due to this, the adjacent threads working on neighbouring cells must skip memory by a stride of 9 to access data. This causes the GPU to process the memory slowly.

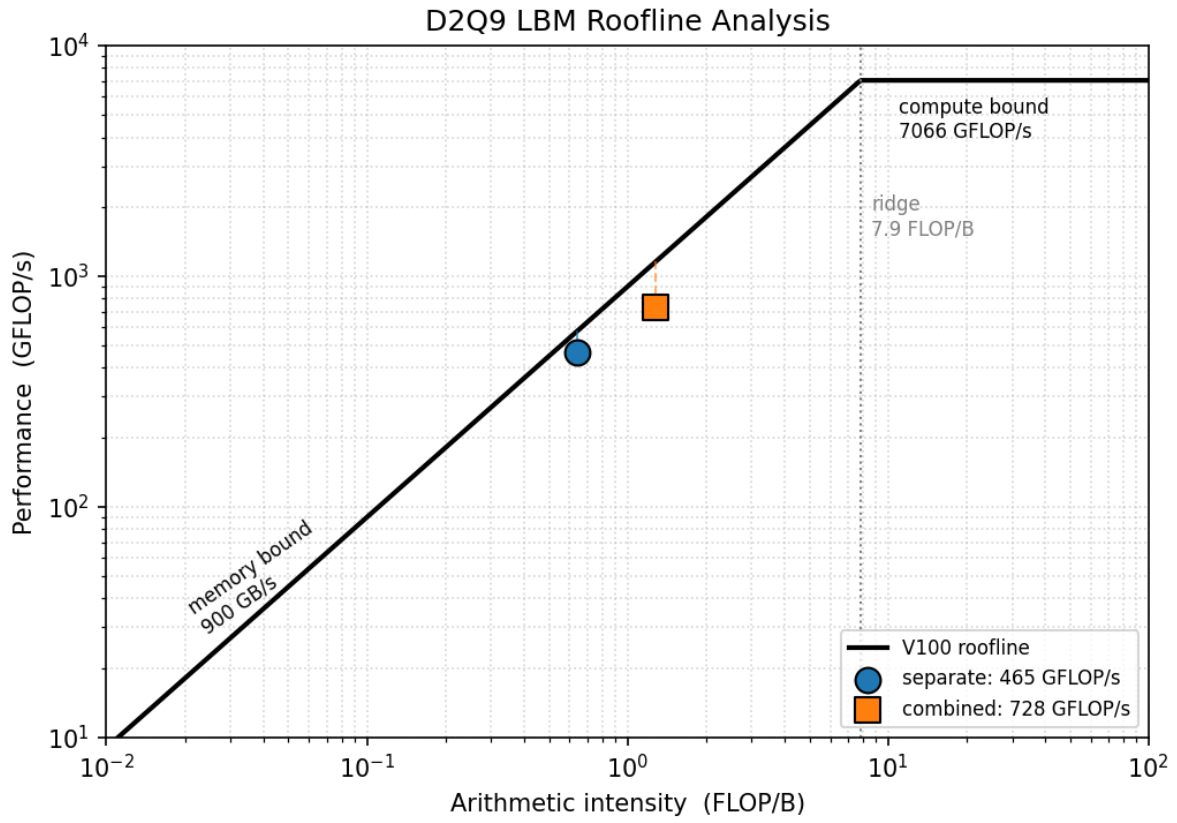


Figure 3: Roofline diagram for the V100. Both the separate (AI = 0.64) and fused (AI = 1.28) kernels lie in the memory-bound region, away from the ridge point (7.85 FLOP/B).

6.4 Kernel Analysis

The combined `collide+stream` kernel achieves 3937 MLUPS in comparison to 2513 MLUPS for separate kernels. The theoretical baseline for each kernel can be computed as:

$$\text{MLUPS}_{\text{th}} = \frac{900 \text{ GB/s}}{\text{Bytes/Cell } \times 10^6} \tag{3}$$

where total bytes transferred for *combined* is 144 B/cell and for *separated* is 288 B/cell giving 6250 MLUPS and 3125 MLUPS as the theoretical upper bound, respectively.

6.5 Roofline Analysis

The D2Q9 collide step requires 185 floating-point operations per cell: 45 for ρ, m_x, m_y accumulation; 5 for u_x, u_y, u^2; and 135 for the nine equilibrium and relaxation updates. Bounce-back and stream perform no floating-point operations. The *separate* kernels read and write all nine distribution functions ($9*2$ `(collide)` $+ 9*2$ `(stream)` doubles = 288B/cell) giving arithmetic intensity AI = 185/288 $\approx$ 0.64 FLOP/B. Combining `collide+stream` reduces to one read and one write which amounts to 144 B/cell, increasing the AI to 1.28 FLOP/B.

The theoretical V100 ridge point is 7066 GFLOP/s $\div$ 900 GB/s = 7.85 FLOP/B. Both kernels are left of this ridge in the memory-bound region (Figure 3). Table 2 summarises the measured performance. The *separate* kernel achieves 465 GFLOP/s whereas combined kernel reaches 728 GFLOP/s. Although *combined* kernel achieves higher MLUPS by reducing data transfer, the achieved bandwidth gets reduced due to poor coalescing. Regardless, both kernels never approach the compute bound region.

7. Conclusion

This study demonstrates that memory bandwidth limits the D2Q9 Lattice Boltzmann Method on both CPU and GPU architectures. For MPI implementation, strong scaling reaches $S \approx 95.8\times$ at $P = 128$ ranks for the 1600×800 grid. The overall speed is sublinear but at high P doubling ranks give super-linear improvement in speedup which is due to higher memory bandwidth due to improved cache hits. Weak scaling shows a saturation limit in single-node efficiency to $\sim 47\%$ at $P = 64$ due to bandwidth saturation. The CUDA implementation achieves 2513 MLUPS (*separate*) and 3937 MLUPS (*combined*) on the V100, $\sim 106\times$ and $\sim 166\times$ over the serial baseline at the largest grid size. Both approaches are memory bandwidth limited, although the *combined* kernel achieves $1.57\times$ higher MLUPS than separate kernels by reducing data transfer but the compute roofline remains unreachable without a change in the algorithm.

8. AI Usage Statement

During this project, Gemini was used for syntax debugging of MPI and CUDA code, generating bash scripts, plotting routines and refinement of report. Additionally, AI assisted with the specific C++ implementation of using `enum classes` *for layout templating and the CUDA probe functions.*